Unit 3: Algorithms and Problem Solving

Complete exercise solutions — MCQs, Short Questions, Long Questions

EXERCISE SOLUTIONS — Unit 3: Algorithms and Problem Solving

Multiple Choice Questions

1. The characteristic of a well-defined problem is:

• a) Ambiguous goals and unclear requirements

• b) Vague processes and inputs

• c) Clear goals, inputs, processes, and outputs

• d) Undefined solutions

Answer: c) Clear goals, inputs, processes, and outputs

2. Complexity class representing problems solvable efficiently by a deterministic algorithm is:

• a) NP

• b) NP-hard

• c) NP-complete

• d) P

Answer: d) P

3. The statement that applies to unsolvable problems is:

• a) They can be solved in polynomial time

• b) They cannot be solved by any algorithm

• c) They are always in NP class

• d) They require exponential time to solve

Answer: b) They cannot be solved by any algorithm

4. The meaning of NP in computational complexity is:

• a) Non-deterministic Polynomial time

• b) Negative Polynomial time

• c) Non-trivial Polynomial time

• d) Numerical Polynomial time

Answer: a) Non-deterministic Polynomial time

5. Search algorithm most efficient for large datasets is:

• a) Bubble Sort

• b) Merge Sort

• c) Selection Sort

studysapiens.com

• d) Quick Sort

Answer: b) Merge Sort

6. A scenario where Dynamic Programming proves most useful is:

• a) Problems without overlapping subproblems

• b) Problems solved by making local choices

• c) Problems with overlapping subproblems and optimal substructure

• d) Problems divided into independent subproblems

Answer: c) Problems with overlapping subproblems and optimal substructure

7. An algorithm that sorts data by stepping through the list and swapping adjacent elements if needed is:

• a) Selection Sort

• b) Quick Sort

• c) Bubble Sort

• d) Merge Sort

Answer: c) Bubble Sort

8. Time complexity of Depth-First Search (DFS) in a graph is:

• a) O(n log n)

• b) O(V)

• c) O(V + E)

• d) O(n)

Answer: c) O(V + E)

9. Best description of time complexity is:

• a) Amount of memory an algorithm needs

• b) Time taken as a function of input size

• c) Efficiency as input size grows

• d) Upper bound of space requirements

Answer: b) Time taken as a function of input size

10. An algorithm with a time complexity of O(n log n) is:

• a) Bubble Sort

• b) Binary Search

• c) Merge Sort

• d) Insertion Sort

Answer: c) Merge Sort

studysapiens.com

Short Questions

1. Differentiate between well-defined and ill-defined problems within the realm of computational problem-solving.

A well-defined problem has clear goals, inputs, processes, and outputs — it can be stated precisely and solved by an algorithm. For example, sorting a list of numbers. An ill-defined problem has ambiguous goals, unclear requirements, or vague inputs — it cannot be fully specified in advance. For example, 'design a good user interface' or 'predict the stock market'. Well-defined problems suit algorithmic solutions; ill-defined problems require human judgement, creativity, and iteration.

2. Outline the main steps involved in the Generate-and-Test method.

The Generate-and-Test method works in three steps: (1) Generate — produce a candidate solution from the set of possible solutions. (2) Test — check whether the candidate satisfies the problem's conditions or constraints. (3) Repeat — if the test fails, generate another candidate and test again. The process continues until a solution passes the test or all candidates are exhausted. It is a brute-force approach used when the solution space is small enough to search.

3. Compare tractable and intractable problems in the context of computational complexity.

Tractable problems can be solved in polynomial time (Class P) — the solution time grows reasonably with input size. Example: sorting with Merge Sort at O(n log n). Intractable problems require super-polynomial or exponential time — the solution time grows too fast to be practical for large inputs. Example: the Traveling Salesman Problem, which is NP-hard. Tractable problems have efficient algorithms; intractable problems have no known efficient algorithm and may require approximations or heuristics.

4. Summarize the key idea behind Greedy Algorithms.

A greedy algorithm makes the locally optimal choice at each step, hoping that these local choices lead to a globally optimal solution. It never revisits a decision once made. Example: the Coin Change problem — always pick the largest coin that fits. Greedy algorithms are simple and fast, but they do not always give the optimal answer; they work only when the problem has the 'greedy-choice property'.

5. Discuss the advantages of using Dynamic Programming.

Dynamic Programming (DP) solves problems by breaking them into overlapping subproblems and storing their results. Advantages: (1) Efficiency — it avoids recomputation by remembering (memoisation) or tabulating results, turning exponential solutions into polynomial ones. (2) Optimality — it guarantees the optimal solution when the problem has optimal substructure. (3) Systematic approach — it provides a clear framework for problems like Fibonacci, knapsack, and shortest paths.

6. Compare the advantages of Breadth-First Search (BFS) with Depth-First Search (DFS) in graph traversal.

BFS explores a graph level by level using a queue. Advantages: it finds the shortest path (in edges) from the start node, and it is good for finding nearby nodes first. DFS explores as far as possible down one branch using a stack before backtracking. Advantages: it uses less memory for deep graphs, and it is good for path-finding and topological sorting. Both have time complexity O(V + E). BFS is better for shortest paths; DFS is better for deep exploration and memory efficiency.

7. Explain the importance of breaking down a problem into smaller components in algorithmic thinking.

Breaking a problem into smaller components (decomposition) is important because: (1) each small part is easier to understand and solve than the whole problem; (2) it allows divide-and-conquer strategies, where each subproblem is solved independently; (3) it makes debugging easier — errors can be isolated to one component; (4) it enables reuse — components can be combined in other solutions. This is the foundation of efficient algorithm design.

8. Identify the key factors used to evaluate the performance of an algorithm.

The key factors are: (1) Time complexity — how the running time grows with input size, expressed in Big O notation (e.g., O(n), O(n log n), O(n^2)). (2) Space complexity — how much memory the algorithm uses. (3) Correctness — whether the algorithm always produces the right output. (4) Worst-case vs. average-case behaviour. (5) Practical factors like constant overhead and cache performance. Time and space complexity are the most important for comparing algorithms.

Long Questions

1. Provide a detailed explanation of why the Halting Problem is considered unsolvable and its implications in computer science.

The Halting Problem asks: given a program and its input, will the program eventually halt (finish) or run forever? Alan Turing proved in 1936 that no general algorithm can solve this for all possible program-input pairs. The proof uses self-reference: assume a halting-checker exists, then build a program that halts exactly when the checker says it will not — a contradiction. Implications: (1) it is impossible to write a perfect virus detector or infinite-loop detector; (2) program analysis tools can only be approximate; (3) it shows fundamental limits of computation — not every well-defined question has an algorithmic answer. It is the classic example of an unsolvable problem.

2. Discuss the characteristics of search problems and compare the efficiency of Linear Search and Binary Search algorithms.

Search problems involve finding a target element in a collection. Characteristics: the collection may be ordered or unordered, and the search must be correct and efficient. Linear Search checks each element one by one — it works on any list but takes O(n) time. Binary Search works only on sorted lists — it repeatedly halves the search space, taking O(log n) time. For a list of 1 million elements, Linear Search may check up to 1 million items, while Binary Search needs at most about 20 comparisons. Binary Search is far more efficient for large sorted datasets, but Linear Search is simpler and works on unsorted data.

3. Discuss the nature of optimization problems and provide examples of their applications in real-world scenarios.

Optimization problems seek the best possible solution among many candidates — the maximum or minimum of some measure. They typically have a set of feasible solutions and an objective function to maximise or minimise. Real-world examples: (1) the Traveling Salesman Problem — find the shortest route visiting all cities; (2) the Knapsack Problem — maximise value within a weight limit; (3) route planning for delivery vehicles; (4) scheduling to minimise waiting time; (5) resource allocation in cloud computing. Optimization problems are often NP-hard, so exact solutions may be impractical for large inputs, and heuristics or approximations are used.

4. Explain the process and time complexity of the Bubble Sort algorithm. Compare it with another sorting algorithm of your choice in terms of efficiency.

Bubble Sort repeatedly steps through the list, comparing adjacent elements and swapping them if they are out of order. After each pass, the largest unsorted element 'bubbles up' to the end. The process repeats until no swaps are needed. Time complexity: O(n^2) in the worst and average cases, O(n) in the best case (already sorted). Comparison with Merge Sort: Merge Sort uses divide and conquer, splitting the list in half and merging sorted halves, with a consistent O(n log n) time complexity. For large datasets, Merge Sort is much faster than Bubble Sort. Bubble Sort is simple and uses little extra memory, making it suitable only for small lists or educational purposes.

5. Discuss the differences between time complexity and space complexity. How do they impact the choice of an algorithm for a specific problem?

Time complexity measures how the running time grows with input size (e.g., O(n), O(n^2)). Space complexity measures how the memory usage grows with input size. They are different resources: an algorithm can be fast but memory-hungry, or slow but memory-light. Impact on choice: (1) if time is critical (real-time systems), choose a low time-complexity algorithm even if it uses more memory; (2) if memory is limited (embedded systems, mobile devices), choose a low space-complexity algorithm; (3) sometimes a trade-off is made — for example, Merge Sort uses extra memory but is faster than in-place sorts. The right choice depends on the constraints of the problem and the environment.